Retrieval: answering from our own documents

The model doesn’t know our runbooks. So we find the right page and hand it over.

60 min | Recorded | Class 9 of 15

9

What this session covers

01

Why retrieval

2 slides

02

Building it

3 slides

03

Where it goes wrong

3 slides

04

Code walkthrough

4 slides

© 2026 Devansh Parapalli

Section 1 of 4

Why retrieval

Class 6 showed it’ll make things up. This is the fix.

01

Retrieve first, then answer

© 2026 Devansh Parapalli

The model has **never read our runbooks**.

Nobody trained it on our runbooks, our tickets or our shipments. It has to be shown them, every time.

© 2026 Devansh Parapalli

Retrieve first, then answer

Also called RAG: retrieval-augmented generation.

01 Find
Search our documents for the few passages that match the question.

02 Hand over
Put those passages in the prompt, numbered.

03 Answer and cite
The model answers only from them, and says which one each point came from.

© 2026 Devansh Parapalli

Section 2 of 4

Building it

Chunk, index, search, answer.

01

A retrieval pipeline is five steps, and only the last one uses an LLM

02

Our first corpus is 8 runbooks, cut into 56 chunks

03

The data class decides which chunks a tool may even see

© 2026 Devansh Parapalli

A retrieval pipeline is five steps, and only the last one uses an LLM

1

Collect the documents, with their data class

2

Cut them into chunks: a step, a paragraph

3

Index each chunk: keywords, vectors, or both

4

Search with the question

5

Answer from the top few, with citations

© 2026 Devansh Parapalli

Our first corpus is 8 runbooks, cut into 56 chunks

Customs documents in Tracklane, InvoiceHub batches, reopening POs, contractor enrolment, scanners, supplier bank changes, payroll variances, badges.

One chunk per numbered step. Small enough to be specific, big enough to make sense on its own.

runbooks	8
chunks	56

© 2026 Devansh Parapalli

The data class decides which chunks a tool may even see

The payroll runbook is LOL Highly Confidential; the bank-change one is Confidential. A tool with a Confidential ceiling never gets the payroll chunks, whatever the question.

Filter at search time, before anything reaches the prompt (class 8).

chunks for Lighthouse	56
for a Confidential tool	49
for an Internal tool	41

© 2026 Devansh Parapalli

Section 3 of 4

Where it goes wrong

Mostly in the search, not in the model.

01

Keyword search finds the step when the words match, and misses it when they don’t

02

Use both kinds of search, because each misses what the other catches

03

Four ways retrieval fails, and how to notice

© 2026 Devansh Parapalli

Keyword search finds the step when the words match, and misses it when they don’t

Words match

“Scanner shows yesterday’s pick list”

- Top hit 0.39: scanner-resync.md, step 3
- Exactly the right step

Words don’t match

“Armadillo says our bank details changed, can I update them from their email?”

- Top hit only 0.12: supplier-bank-change.md, closing line
- Missed step 1: “Never act on an email”

© 2026 Devansh Parapalli

Use both kinds of search, because each misses what the other catches

Keyword
Great at IDs and exact terms: `IH-260615-11`, `ERR-IH-0413`, “Harbor”.

Vectors (class 5)
Great at meaning: “update bank details from their email” finds “never act on an email”.

Both
Run the two, merge the results, keep the top few. Called hybrid search.

© 2026 Devansh Parapalli

Four ways retrieval fails, and how to notice

Wrong chunk	The answer cites a passage that doesn’t say that. Check citations on a sample every week.
Stale document	The runbook is from before the process changed. Show the “last reviewed” date with every answer.
Nothing matches	It should say “not in the runbooks”, not improvise. Test that on purpose.
Too much context	Twenty chunks bury the right one. Three to five is usually enough.

© 2026 Devansh Parapalli

Section 4 of 4

Code walkthrough

Chunk our runbooks, search them, answer with citations.

01

Chunking is one split per paragraph or numbered step, tagged with the source

02

The prompt tells it to answer only from the extracts, and to say so when they don’t help

03

Where today sits on the card

04

What to do next

© 2026 Devansh Parapalli

Chunking is one split per paragraph or numbered step, tagged with the source

code/class09/rag.py

```
for path in sorted(RUNBOOKS.glob("*.md")):
    text = path.read_text(encoding="utf-8")
    cls = re.search(r"Class: (LOL [A-Za-z ]+?)\.", text).group(1)
    if CEILING[cls] > CEILING[tool_limit]:
        continue                                  # this tool may not see this runbook
    for part in re.split(r"\n\s*\n|\n(?=\d+\. )", text):
        out.append({"source": path.name, "text": part.strip()})
```


© 2026 Devansh Parapalli

The prompt tells it to answer only from the extracts, and to say so when they don’t help

code/class09/rag.py

```
hits = search(question)
context = "\n\n".join(f"[{i + 1}] ({c['source']}) {c['text']}" for i, (_, c) in
enumerate(hits))
return ask("Answer only from the numbered runbook extracts. Cite them like [1]. "
           "If they don't answer the question, say so.\n\n"
           f"{context}\n\nQuestion: {question}", temperature=0)
```


© 2026 Devansh Parapalli

Where today sits on the card

1. A script	The steps are known and don’t change.
2. A rule	The decision is known and must be explainable line by line.
3. Classical ML	Labelled history, and the input is a table or short text.
4. A small trained model	Lots of text, images or audio, enough labels, cost per call matters.
5. An LLM with a prompt	Varied language in and out, few labels, someone checks the output.
6. An LLM plus retrieval, tools or agents	**Needs private knowledge, has to act, or takes several steps.**
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Ask `rag.py` three questions from the handwritten tickets and check every citation.
2. Swap the keyword search for embeddings and rerun the bank-change question.
3. Read the class 10 pre-reading on function calling.

Questions after class: the course channel.

© 2026 Devansh Parapalli